

---

# COMP0037 COURSEWORK 2

---

A PREPRINT

**Group Name: 11**  
Department of Computer Science  
University College London  
London, WC1E 6BT

April 30, 2025

## 1 Part 1

### Q1(a)

On-policy Monte Carlo (MC) methods estimate the state-value function  $v_\pi(s)$  by generating complete episodes directly under a given target policy  $\pi$ . For each time step  $t$ , the full return is computed as:

$$G_t = R_{t+1} + \gamma G_{t+1}, \quad (1)$$

The estimate  $v_\pi(s)$  is formed by averaging either the first visit of  $s$  in each episode (first-visit MC) or every visit (every-visit MC).

In contrast, off-policy MC uses a separate behaviour policy  $b$  to collect episodes while evaluating a target policy  $\pi$ . Each return  $G_t$  is weighted by the correction factor:

$$\rho_{t:T-1} = \frac{\prod_{k=t}^{T-1} \pi(A_k | S_k)}{\prod_{k=t}^{T-1} b(A_k | S_k)}, \quad (2)$$

The  $v_\pi(s)$  is estimated as"

$$v_\pi(s) = \frac{\sum \rho_{t:T-1} G_t \mid S_t = s}{\sum \rho_{t:T-1}}, \quad (3)$$

This estimation is valid under the coverage assumption  $b(a \mid s) > 0$  whenever  $\pi(a \mid s) > 0$ .

The main difference between MC and model-based approaches is that MC eliminates the requirement of an existing model. Model-based approaches learn from a perfect rewards and state transitions model and apply value or policy iterations to compute  $v^*(s)$  or  $\pi^*(s)$ . In contrast, MC approaches only use sample returns from the completed episodes to estimate  $v_\pi(s)$ . Also, instead of sweeping over the entire state space, MC only updates values for states visited in each episode, reducing the amount of data since some states could rarely be encountered during training. Another difference is that model-based approaches update the value after every transition. At the same time, MC must wait until the end of an episode to complete the full return and update the value estimates.

Some potential issues of MC learning are:

- The learning progress could be very slow due to random exploration and noisy updates
- Learning only take place once an episode has been completed

### Q1(b)

To compare the performance of different Monte Carlo approaches, we compute the Mean Squared Error (MSE) between the estimate  $v_\pi(s)$  and the ground-truth  $v^*(s)$  after  $N$  episodes:

$$\text{MSE}(N) = \frac{1}{M} \sum_{s \in \mathcal{S}} (\hat{v}_N(s) - v^*(s))^2, \quad (4)$$

where:

- $\mathcal{S}$  is the set of all non-obstructed states in the grid.
- $M = |\mathcal{S}|$  is the number of states compared.
- $\hat{v}_N(s)$  is the value estimate produced by a given MC algorithm after  $N$  episodes.
- $v^*(s)$  is the true value from the PolicyEvaluator.

A low MSE indicates that the MC estimate is close to the ground truth, representing a good performance.

When the number of episodes  $N = 100$ , which is the default value given in the code, the final policy is presented in Figure 1,2,3 and 4, and the MSE of each approach is shown in Table 1:

Figure 1: On-Policy MC First Visit

Figure 2: On-Policy MC Every Visit

Figure 3: Off-Policy MC First Visit

Figure 4: Off-Policy MC Every Visit

|                    | On-Policy | Off-Policy |
|--------------------|-----------|------------|
| <b>First Visit</b> | 0.4766    | 998.9194   |
| <b>Every Visit</b> | 0.1390    | 298.6244   |

Table 1: Mean Squared Error (MSE) after 100 episodes

In general, on-policy MC performs better than off-policy, which is expected as on-policy MC generates samples directly under the target policy  $\pi$ . The returns of it are unweighted samples of  $v_\pi$ . While off-policy, MC introduced the behaviour policy  $b$ , which has weighted the return. This makes the variance of the returns significantly higher compared to on-policy. As the number of episodes is 100, a relatively small number, the influence of the variance is significant.

Our result shows that the every-visit approach outperforms the first-visit approach when the number of episodes is 100. Although the every-visit approach introduces bias due to repeatedly including the same sub-path, as the policy is trained in a grid environment with simple actions, the possibility of re-visiting states within episodes is limited, which makes this bias insignificant.

Table 2 gives the full MSE of different approaches with varying number of episodes. Full policy visualisation is shown in appendix.

| Episodes | On-Policy FV | On-Policy EV | Off-Policy FV | Off-Policy EV |
|----------|--------------|--------------|---------------|---------------|
| 50       | 0.6551       | 0.1448       | 1271.4364     | 1088.7661     |
| 100      | 0.4766       | 0.1390       | 998.9194      | 298.6244      |
| 200      | 0.2760       | 0.0290       | 145.2951      | 298.8113      |
| 1000     | 0.3578       | 0.0098       | 1.2015        | 1.5826        |

Table 2: Full MSE for different number of episodes

As the number of episodes increases, all approaches perform better in MSE. On-policy consistently performs better than off-policy. Moreover, for the on-policy approach, Every visit constantly outperforms the First visit approach, which

aligns with the result when  $N = 100$ . The key difference is that starting from  $N = 200$ , off-policy FV outperforms off-policy EV and keeps the advantage until  $N = 1000$ . For a larger number of episodes, the bias introduced by EV starts to become more significant, which allows FV to generate a better result.

### Q1(c)

Target policy  $\pi(A|S)$  is the policy that the robot runs after the training has been carried out, while behaviour policy  $b(A|S)$  is what the robot uses to learn what the target policy is. In off-policy MC predictions,  $b(A|S) \neq \pi(A|S)$  and  $b(A|S)$  is set as an  $\epsilon$ -Greedy exploration to enhance the robot to explore for optimal policies.  $\epsilon_b$  is useful in defining  $b(A|S)$ , where:

$$b(a | s; \epsilon_b) = \begin{cases} 1 - \epsilon_b + \frac{\epsilon_b}{|A(s)|}, & \text{if } a = \arg \max_{a'} q_\pi(s, a') \\ \frac{\epsilon_b}{|A(s)|}, & \text{otherwise.} \end{cases} \quad 0 \leq \epsilon_b \leq 1 \quad (5)$$

Theoretically, setting  $\epsilon_b$  close to or equal to 1 makes the behaviour policy perfectly uniform,  $b(a | s) = 1/|A(s)|$  for every action. If the target policy is nearly greedy,  $\pi(a^* | s) \approx 1$ , the importance-sampling ratio at each greedy step becomes  $\rho_t = \pi(a^* | s)/b(a^* | s) = |A(s)|$ . Therefore, the cumulative weight over an episode grows like  $|A(s)|^{n_g}$ , where  $n_g$  is the number of greedy actions encountered. This quantity explodes exponentially, quickly overflowing the floating-point range and causing the off-policy Monte-Carlo run to crash.

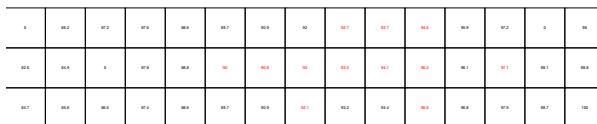
At the other extreme, an extremely small  $\epsilon_b$  makes the behaviour policy almost identical to the greedy target. Individual ratios  $\rho_t$  is close to 1. Although numerical instability is avoided, the non-greedy actions are selected rarely. Since those rare excursions are essential for accurate importance sampling, the effective sample size is reduced; many more episodes are needed before the weighted returns average out, leading to noticeably slow convergence of the value estimates.

To verify our interpretation, a range of  $\epsilon_b$  has been selected: 0.01, 0.1, 0.2, 0.5, 1.0. The number of episodes is set to 500, and for every 50 episodes, the MSE between the policy generated and the ground truth are recorded for quantitative analysis. Table 3 shows the quantitative performance of each  $\epsilon_b$ .

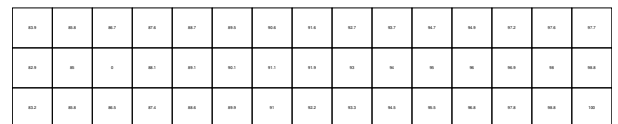
**Table 3:** Performance of Off-Policy Monte Carlo under Different  $\epsilon_b$

| Episode | Mean Squared Error (MSE) |                    |                    |                    |                    |
|---------|--------------------------|--------------------|--------------------|--------------------|--------------------|
|         | $\epsilon_b = 0.01$      | $\epsilon_b = 0.1$ | $\epsilon_b = 0.2$ | $\epsilon_b = 0.5$ | $\epsilon_b = 1.0$ |
| 50      | 213.60                   | 366.09             | 155.62             | 4696.21            | 5686.57            |
| 100     | 213.34                   | 366.07             | 155.58             | 3767.54            | 5481.81            |
| 150     | 213.34                   | 365.89             | 2.92               | 3197.43            | 5481.80            |
| 200     | 213.30                   | 2.49               | 2.64               | 2480.13            | 5481.77            |
| 250     | 213.17                   | 2.49               | 2.33               | 2480.14            | 5481.77            |
| 300     | 213.22                   | 2.51               | 2.38               | 2480.18            | 5284.48            |
| 350     | 213.14                   | 2.51               | 1.66               | 2480.22            | 5082.23            |
| 400     | 213.05                   | 2.28               | 1.67               | 2295.51            | 5082.18            |
| 450     | 1.89                     | 2.35               | 1.48               | 1948.04            | 5082.18            |
| 500     | 1.89                     | 2.32               | 1.46               | 1948.02            | 5082.18            |

From table 3, the optimal result is observed when  $\epsilon_b = 0.1$  and  $\epsilon_b = 0.2$ . With a high initial MSE,  $\epsilon_b = 0.2$  converges at around episode 100 (155.58 to 2.92) and  $\epsilon_b = 0.1$  converges at episode 150 (365.89 to 2.49). The MSE of both training was stabilized after convergence and remained in a desired range. Qualitatively, from figure 9 and 7, we observed two unwanted zero for  $\epsilon_b = 0.1$  and one for  $\epsilon_b = 0.2$  when the number of episodes is 100, showing that the system is still converging. After  $N = 500$ , figure 6 and 8 shows that both policies converged to a similar policy to the ground truth.



**Figure 5:**  $\epsilon_b = 0.1(N=100)$



**Figure 6:**  $\epsilon_b = 0.1(N=500)$

|       |       |       |       |       |       |       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 0     | 182.7 | 187.2 | 182.7 | 182.7 | 183.4 | 183.9 | 183.4 | 183.9 | 182.2 | 182.2 | 182.2 | 187.2 | 187.2 | 18    |
| 183.4 | 183.4 | 0     | 187.4 | 182.2 | 182.2 | 183.4 | 183.9 | 183.4 | 183.9 | 182.2 | 182.2 | 187.2 | 187.2 | 183.4 |
| 183.4 | 187.2 | 187.2 | 182.2 | 183.4 | 183.4 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 187.2 | 187.2 | 183.4 |

Figure 7:  $\epsilon_b = 0.2(N=100)$ 

|       |       |       |       |       |       |       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 182.2 | 182.2 | 182.2 | 182.2 | 187.2 | 182.2 | 18    | 182.2 | 182.2 | 182.2 | 187.2 | 182.2 | 182.2 | 187.2 | 182.2 |
| 18    | 182.2 | 0     | 187.2 | 187.2 | 18    | 182.2 | 182.2 | 182.2 | 182.2 | 18    | 182.2 | 182.2 | 18    | 182.2 |
| 182.2 | 182.2 | 182.2 | 187.2 | 187.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 187.2 | 182.2 | 18    |

Figure 8:  $\epsilon_b = 0.2(N=500)$ 

When  $\epsilon_b = 0.01$ , the initial MSE is high (213.60), which is expected. However, the MSE remains at a high value, around 213, with no clear intention to converge until episode 450, where a rapid drop of MSE (213.05 to 1.89) occurs. This shows that the system can reach convergence but requires additional time due to the small  $\epsilon_b$ . Qualitatively, from figure 9, we observed a single unwanted 0 in the policy generated. It consistently exists until episode 450, shown in figure 10. The policy in episode 450 is already close to the ground truth, showing that the prediction is effective. This aligns with our theoretical knowledge that an extremely small  $\epsilon_b$  leads to convergence but requires extra time.

|       |       |       |       |       |       |       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 183.4 | 183.4 | 187.2 | 187.2 | 187.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 187.2 | 187.2 | 18    |
| 183.4 | 183.4 | 0     | 187.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 187.2 | 187.2 | 183.4 |
| 183.4 | 182.2 | 182.2 | 187.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 187.2 | 187.2 | 183.4 |

Figure 9:  $\epsilon_b = 0.01(N=100)$ 

|       |       |       |       |       |       |       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 182.2 | 182.2 | 182.2 | 187.2 | 182.2 | 182.2 | 18    | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 187.2 | 182.2 | 18    |
| 182.2 | 182.2 | 0     | 182.2 | 182.2 | 18    | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 187.2 | 182.2 | 182.2 |
| 182.2 | 182.2 | 182.2 | 187.2 | 182.2 | 18    | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 187.2 | 18    | 182.2 |

Figure 10:  $\epsilon_b = 0.01(N=450)$ 

When  $\epsilon_b = 0.5$  and  $\epsilon_b = 1.0$ , the training is ineffective. Both training starts with an extremely high initial MSE (4696.21 for  $\epsilon_b = 0.5$  and 5686.57 for  $\epsilon_b = 1.0$ ). Although with a decreasing trend, both MSEs stay at a high value even after 500 episodes. Qualitatively, from figure 11 and 13, we observed that both policies at episode 100 have the majority of the states zero, showing that most of them are not properly updated, which does not get much better after 500 episodes, shown in figure 12 and 14.

|   |   |   |   |   |       |       |       |       |       |       |       |       |       |       |
|---|---|---|---|---|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 0 | 0 | 0 | 0 | 0 | 0     | 0     | 0     | 0     | 0     | 0     | 0     | 183.4 | 18    | 187.2 |
| 0 | 0 | 0 | 0 | 0 | 0     | 0     | 0     | 0     | 183.4 | 18    | 183.4 | 182.2 | 18    | 183.4 |
| 0 | 0 | 0 | 0 | 0 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 187.2 | 182.2 | 183.4 |

Figure 11:  $\epsilon_b = 0.5(N=100)$ 

|   |   |       |       |       |       |       |       |       |       |       |       |       |       |       |
|---|---|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 0 | 0 | 0     | 0     | 0     | 182.2 | 182.2 | 0     | 182.2 | 182.2 | 187.2 | 182.2 | 182.2 | 182.2 | 187.2 |
| 0 | 0 | 0     | 0     | 0     | 0     | 0     | 182.2 | 18    | 182.2 | 182.2 | 182.2 | 182.2 | 187.2 | 183.4 |
| 0 | 0 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 18    | 182.2 | 182.2 | 182.2 | 182.2 | 182.2 | 183.4 |

Figure 12:  $\epsilon_b = 0.5(N=500)$ 

|   |   |   |   |   |   |   |   |   |   |   |       |       |       |       |
|---|---|---|---|---|---|---|---|---|---|---|-------|-------|-------|-------|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0     | 182.2 | 182.2 | 187.2 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 182.2 | 182.2 | 182.2 | 182.2 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 18    | 187.2 | 182.2 | 183.4 |

Figure 13:  $\epsilon_b = 1.0(N=100)$ 

|   |   |   |   |   |   |   |   |   |   |   |       |       |       |       |
|---|---|---|---|---|---|---|---|---|---|---|-------|-------|-------|-------|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0     | 182.2 | 182.2 | 187.2 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 182.2 | 182.2 | 182.2 | 182.2 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 18    | 187.2 | 182.2 | 183.4 |

Figure 14:  $\epsilon_b = 1.0(N=500)$ 

#### Q1(d)

One of the simplest Temporal Difference Learning (TDL) algorithms is TD(0). It is a one-step TD method that corrects the current value estimate for a state using the immediate reward and the estimate of the next state. Given a target policy  $\pi$ , follow the trajectory  $S_0, A_0, R_1, S_1, \dots$  and after every step update

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)], \quad (6)$$

where  $\alpha \in (0, 1]$  is the learning rate and  $\gamma$  the discount factor. This bootstraps by using the current estimate  $V(S_{t+1})$  in place of a full return, so it learns online without waiting for an episode to finish. Because the target spans exactly one step, updates can be performed online after every transition and need no model of the dynamics.

For prediction, TD algorithms are typically on-policy they use the actual transition generated by the policy being evaluated. This is good for stability and low-variance learning, but bad if one needs to evaluate a policy without executing it, because off-policy TD requires additional machinery such as importance sampling or specialised algorithms.

| Method                          | Advantages                                                                                                                                                                                                                   | Disadvantages                                                                                                                                                                                      |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Temporal Difference (TD)</b> | <ul style="list-style-type: none"> <li>• Online update every step</li> <li>• Works for continuing &amp; non-stationary tasks</li> <li>• Lower variance; tiny memory footprint</li> <li>• Constant cost per update</li> </ul> | <ul style="list-style-type: none"> <li>• Small bootstrap bias</li> <li>• Typically on-policy only</li> <li>• Can diverge with poor step-size / features</li> </ul>                                 |
| <b>Monte Carlo (MC)</b>         | <ul style="list-style-type: none"> <li>• Unbiased value estimates</li> <li>• Conceptually simple (full-return average)</li> <li>• Off-policy possible via importance sampling</li> </ul>                                     | <ul style="list-style-type: none"> <li>• Must wait for episode to finish</li> <li>• High variance, especially with long episodes</li> <li>• Memory/computation grow with episode length</li> </ul> |

**Table 4:** Comparison of TD and MC prediction methods.**Q1(e)**

Below shows the pseudocode for TD(0) algorithm:

**Algorithm 1** Tabular TD(0) for estimating  $v_\pi$ 

```

1: Input: the policy  $\pi$  to be evaluated
2: Algorithm parameter: step size  $\alpha \in (0, 1]$ 
3: Initialize  $V(s)$ , for all  $s \in S^+$ , arbitrarily except that  $V(\text{terminal}) = 0$ 
4: for each episode do
5:   Initialize  $S$ 
6:   repeat
7:      $A \leftarrow$  action given by  $\pi$  for  $S$ 
8:     Take action  $A$ , observe  $R, S'$ 
9:      $V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$ 
10:     $S \leftarrow S'$ 
11:   until  $S$  is terminal
12: end for
```

The implementation begins by initializing a timestep counter ‘t’ to track our position in the episode.

```

1 t = 0
2 while True:
3     try:
```

Next, the code retrieves the current state and attempts to access the next state. This represents our position in the environment and implements the state transition dynamics required by TD(0).

```

1 current_state = episode.state(t)
2 next_state = episode.state(t + 1)
```

If we successfully access both states, it means we haven’t reached the end of the episode. The code then extracts the coordinates of these states, which is necessary for our tabular representation of the value function.

```

1 current_coords = current_state.coords()
2 x_cell_coord, y_cell_coord = current_coords[0], current_coords[1]
3 next_coords = next_state.coords()
4 next_x, next_y = next_coords[0], next_coords[1]
```

After obtaining the state representations, the implementation retrieves the reward received during this transition.

```

1 reward = episode.reward(t)
```

The code then accesses the current value estimates for both the current and next states. .

```

1 current_value = self._v.value(x_cell_coord, y_cell_coord)
2 next_value = self._v.value(next_x, next_y)

```

With all necessary values obtained, the implementation now calculates the TD target and TD error.

```

1 td_target = reward + self._gamma * next_value
2 td_error = td_target - current_value

```

Using the TD error, the code updates the value function. This is the core of the TD(0) learning process.

```

1 new_v = current_value + self._alpha * td_error
2 self._v.set_value(x_cell_coord, y_cell_coord, new_v)

```

After updating the value function, the implementation increments the timestep counter to move to the next state-action pair.

```

1 t += 1

```

Finally, the code handles the termination condition. If an exception occurs when trying to access a state beyond the end of the episode, it means we've reached a terminal state. In this case, the code breaks out of the loop.

```

1 except Exception:
2     break

```

#### Q1(f)

A range of values of  $\alpha$  is tested (0.01, 0.1, 0.2, 0.5, 1.0). The final number of episode is set to 400. Similar to previous approaches, for every 40 episode, the MSE of the current policy against the ground truth is computed for quantitative analysis of policy performance. Table 5 summarised the result and visualisations of predictions are shown in appendix.

**Table 5:** TD(0) Performance Under Different Learning Rate  $\alpha$

| Episode | Mean Squared Error (MSE) |                |                |                |                |
|---------|--------------------------|----------------|----------------|----------------|----------------|
|         | $\alpha = 0.01$          | $\alpha = 0.1$ | $\alpha = 0.2$ | $\alpha = 0.5$ | $\alpha = 1.0$ |
| 40      | 2348.29                  | 0.63           | 0.31           | 1.01           | 2.35           |
| 80      | 649.42                   | 0.27           | 0.46           | 1.21           | 3.57           |
| 120     | 138.26                   | 0.24           | 0.25           | 0.61           | 3.23           |
| 160     | 21.27                    | 0.10           | 0.25           | 0.19           | 2.10           |
| 200     | 4.20                     | 9.29           | 0.31           | 0.73           | 2.24           |
| 240     | 0.63                     | 0.08           | 0.09           | 0.49           | 3.04           |
| 280     | 0.19                     | 0.29           | 0.48           | 0.73           | 2.87           |
| 320     | 0.11                     | 0.38           | 0.13           | 0.84           | 3.19           |
| 360     | 0.05                     | 0.31           | 0.18           | 0.81           | 3.08           |
| 400     | 0.07                     | 0.30           | 0.47           | 0.64           | 3.16           |

We observed that for extremely small  $\alpha$  (0.01), the initial MSE is very large (2348.29) and takes around 240 episodes to reach a more optimal range. For  $\alpha$  between 0.1 to 0.5, the initial MSE is small (0.63, 0.31, 1.01), and all converges fast at approximately episode 160. This shows that a larger  $\alpha$  usually speeds up the convergence rate.

The value of final solutions differs but to a very small extent.  $\alpha = 0.01$  generates the best final policy at  $N = 400$ , with MSE of 0.07. For  $\alpha$  between 0.1 to 0.5, we notice that the MSE of the final policy generated after 400 episodes gets slightly higher when  $\alpha$  increases but still remains in an optimal range ( $\leq 1$ ), which is likely caused by existing residual error. Thus, we assume that there is no significant difference between the predictions. This fits our expectation that TD Learning guarantees optimal solutions.

However, when  $\alpha = 1.0$ , the MSE value constantly floats around 2.5 and reaches a final value of 3.16, significantly larger than other  $\alpha$  values. The final values are not converging to the ground truth. In this case, every new sample overwrites the previous estimate, so the running value never gets a chance to average out the randomness, leading to an unstable solution.

**Q1(g)**

To extract a policy  $\pi$  from a given state-value function  $v_\pi(s)$ , we perform a single policy-improvement pass of Generalized Policy Iteration (GPI). This process requires:

- The discount factor  $\gamma$  used when learning the original value function
- Either:
  - A dynamics model  $(P, R)$  where  $P$  represents transition probabilities and  $R$  represents rewards, or
  - Access to transition samples in the form  $\langle s, a, r, s' \rangle$

The extraction process follows two steps:

1. Convert state values to state-action values:

$$q_\pi(s, a) = \mathbb{E}[r + \gamma v_\pi(s') | s, a] \quad (7)$$

With a dynamics model, we compute this directly:

$$q_\pi(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma v_\pi(s')] \quad (8)$$

With transition samples, we estimate this expectation empirically.

2. Define the policy by selecting actions with highest Q-values:

$$\pi(s) = \arg \max_a q_\pi(s, a) \quad (9)$$

For stochastic policies, we can use  $\epsilon$ -greedy or softmax variants.

After extracting this baseline policy, further optimization is possible through:

- **Monte Carlo control:** Uses complete episodes to refine  $q$ -values:
  - On-policy: Updates the same policy being followed
  - Off-policy: Uses importance sampling to learn while following a different behavior policy
- **Temporal Difference control:** Uses bootstrapped estimates for faster learning:
  - SARSA (on-policy): Updates using  $r + \gamma Q(s', a')$
  - Q-learning (off-policy): Updates using  $r + \gamma \max_{a'} Q(s', a')$

These methods continually alternate between evaluation and improvement steps, gradually moving the extracted policy toward optimality rather than leaving it as is.

**Q2(a)**

**We introduced the off-policy method in Q1(a):** An off-policy method learns about a target policy  $\pi$ , while following a different behavior policy  $b$ . The behavior policy is used to generate the data, but the learning update is made with respect to the target policy. The calculation of estimated value function was shown in *Equ 3*.

**We introduced the TD(0) method in Q1(d):** Temporal-Difference learning with zero-step lookahead is a method that updates the value of a state using the observed reward and the estimated value of the next state. It performs updates after every step without waiting for the end of the episode. The update rule was shown in *Equ 6*.

**Q-learning** is a Temporal-Difference (TD) **control algorithm** that estimates the optimal action-value function  $q_*(s, a)$  independently of the policy being followed. Its update rule for TD(0) is given by:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[ R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right], \quad (10)$$

where:

- $Q(S_t, A_t)$ : current estimate of the action-value.
- $R_{t+1}$ : reward after taking action  $A_t$  in state  $S_t$ .
- $\gamma$ : discount factor.
- $\alpha$ : learning rate.
- $\max_a Q(S_{t+1}, a)$ : best estimated value for the next state.

**From Equ 10, we find this rule makes use of two policies:**

- The **behavior policy**, which is used to generate episodes. This is typically an  $\epsilon$ -soft policy that explores by occasionally choosing random actions. We denote the actions in this policy as:  $A$ .
- The **target policy**, which is used to evaluate and update the action value function. In Q-learning, this is the greedy policy that always selects the action with the highest  $Q$  value, which was denoted as  $a$ .

**Since the action chosen during learning ( $A_t$ ) may differ from the action used in the update ( $\arg \max_a Q$ ), the method is off-policy by nature. This separation allows Q-learning to learn the optimal policy while exploring with a more exploratory behavior.**



**Q2(b)**

Our implementation follows the core structure and update logic outlined in the pseudocode presented by **Sutton and Barto**:

**Q-learning (off-policy TD control) for estimating  $\pi \approx \pi_*$** 

**Algorithm parameters:** step size  $\alpha \in (0, 1]$ , small  $\varepsilon > 0$

**Initialize**  $Q(s, a)$  arbitrarily for all  $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$  except that  $Q(\text{terminal}, \cdot) = 0$

**Loop** for each episode:

- Initialize  $S$
- **Loop** for each step of episode:
  - Choose  $A$  from  $S$  using a policy derived from  $Q$  (e.g.,  $\varepsilon$ -greedy)
  - Take action  $A$ , observe reward  $R$ , next state  $S'$
  - $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
  - $S \leftarrow S'$
- until  $S$  is terminal

Below is a step-by-step explanation of how each element of the pseudocode maps to the actual Python code.

- **Set the initial state  $S$ , action  $A$ , reward  $R$  at the start of the episode:**

```

1  # state
2  s = episode.state(0)
3  # state coordinates
4  coords = s.coords()
5  # reward
6  reward = episode.reward(0)
7  # action
8  a = episode.action(0)
9

```

- **Loop for each step of episode:**

```

1  for step_count in range(1, episode.number_of_steps()):
2

```

- **Get next state  $S'$  and its coordinates**

```

1  s_next = episode.state(step_count)
2  coords_next = s_next.coords()
3

```

- **Find the maximum estimated Q-value in the next state  $\max_a Q(S', a)$ :**

```

1  # find all actions
2  valid_actions = list(LowLevelActionType[:LowLevelActionType.
NUMBER_OF_ACTIONS])
3  # greedy-policy, find the best state-action value
4  max_q_next = max([self._Q[coords_next][a_prime] for a_prime in
valid_actions])
5

```

- **Calculate the new Q-value  $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ :**

```

1  # Define the current Q-value Q(S,A)
2  current_q = self._Q[coords][a]
3  # Calculate the new Q-value
4  new_q = current_q + self._alpha * (reward + self._gamma * max_q_next -
current_q)
5

```

- **Update the the grid:**

```

1  self._update_q_and_policy(coords, a, new_q)
2

```

- Set the current state, action, reward for the next episode  $S \leftarrow S', A \leftarrow A', R \leftarrow R'$ :

```

1     reward = episode.reward(step_count)
2     s = episode.state(step_count)
3     coords = s.coords()
4     a = episode.action(step_count)
5

```

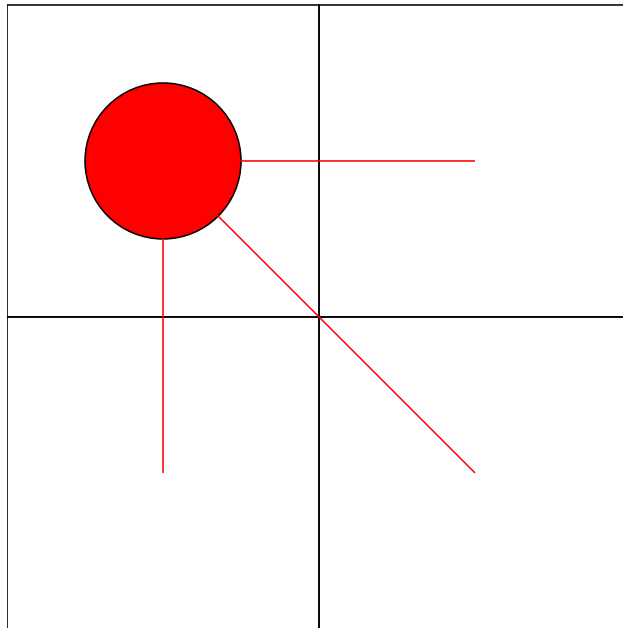
- Set the final Q-value =  $R$  (action is terminate):

```

1     new_q = reward
2     self._update_q_and_policy(coords, a, new_q)
3

```

This completes one full pass of the Q-learning update over a single episode.



**Figure 15:** Illustration of agent action selection in a grid environment.

**Figure 15** is a visual representation of the strategies learned after Q-learning convergence. The red circle is the target endpoint; the red lines are the greedy actions (strategies) chosen by the agent in different states, guiding it on how to move to the endpoint. This has proved that q-learner succeeded in getting the optimal policy.

**Q2(c)**

**Before analysing the performance of q-learner, we need to understand the current scenario:**

- This is a **20\*7 grid environment**. At the top and bottom of the map are **walls**, and there will be four **chairs** next to the bottom side wall. At the top right of the map, there will be four **termination points**. These are get from the function `corridor_scenario()` in the provided `scenarios.py` file:

```

1 def corridor_scenario():
2     airport_map = AirportMap("Corridor Scenario", 20, 7)
3
4     _set_block_to_single_type(airport_map, MapCellType.WALL, (0, 0), (19, 0))
5     _set_block_to_single_type(airport_map, MapCellType.WALL, (0, 6), (19, 6))
6     _set_block_to_single_type(airport_map, MapCellType.CHAIR, (2, 1), (5, 1))
7
8     for y in range(3,7):
9         airport_map.add_robot_end_station(19, y, 100)
10
11     return airport_map, 450
12

```

- Although model-free methods (Q-learning) do not learn the model of the environment, but they still critically depend on the **reward function**. The agent must observe rewards from the environment to learn value functions or policies. So, in order to analyse the performance of the agent, we still need to know the reward and punishment mechanism of this scenario first: (These are get from `next_state_and_reward_distribution()` method of the provided `LowLevelEnvironment(Env)` class:

1. **Terminate action** ( $a = \text{TERMINATE}$ ):

- If the current cell is terminal, the next state is None, reward is the terminal reward (100), with *probability* = 1.

2. **No-op action** ( $a = \text{NONE}$ ):

- The agent remains in the same cell, receives a penalty of  $-1$ , with probability 1.

3. **Driving actions** ( $a \in \{0, \dots, 7\}$ ):

- The intended direction  $a$  has probability  $p$ , and adjacent directions  $(a - 1)$  and  $(a + 1)$  have probability  $q$ .
- If the next position  $(x', y')$  is outside the grid bounds, the agent stays in place with a penalty of  $-1$ .
- If the next cell is obstructed:
  - \* If it is a baggage claim cell (not in this scenario), reward is  $-10$ .
  - \* Otherwise (**wall, chairs**), reward is  $-1$ .
- Otherwise, the transition is successful and the reward is the negative movement cost:

```

1 def compute_transition_cost(self, last_coords, current_coords):
2
3     # Compute the basic Euclidean cost
4     dX = current_coords[0] - last_coords[0]
5     dY = current_coords[1] - last_coords[1]
6     L = math.sqrt(dX * dX + dY * dY)
7
8     return L
9

```

Here are the result policy and value functions for this scenario:

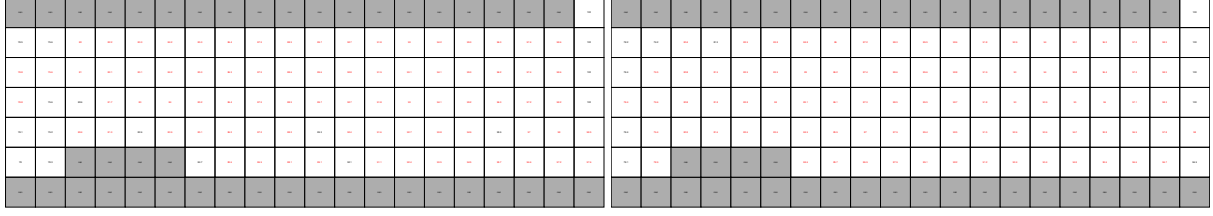


Figure 16: Value functions for two separate experiments

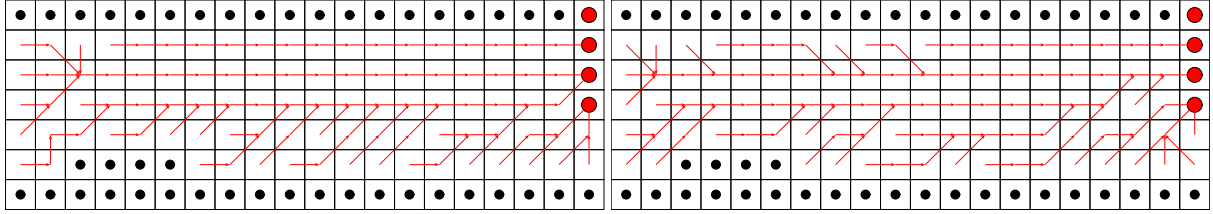


Figure 17: Policy for two separate experiments

**Figure 16** shows the value function is consistent with the reward and punishment mechanism we derived earlier. Positions farther away from the end point will always have lower values than closer positions because the robot needs more movement cost. Even if the distance is the same, positions next to obstacles will have lower values because the randomness of the robot's movement may result in more penalties for crashing obstacles. But the robot is unaware of this because it learns only through the value function not the reward function.

**Figure 17** shows the final policy learned by the Q-learning algorithm in a grid environment for two separate experiments. The red circles represent terminal goal states, and the red arrows indicate the greedy actions chosen from each state based on the learned Q-values.

It is clear that the two experiments did not produce exactly the same results. But that just reflects the Q-learning's **off-policy** nature. Because agent's behaviour policy is  $\epsilon$ -soft - always keeping exploring stochastically. If there are not enough iterations and the final result does not converge completely, which might lead to a slight difference between the final value function and the policy for each experiment. And there is also randomness in the robot's actions.

However, the overall trend consistently illustrates a near-optimal routing strategy. Despite the randomness in individual trajectories, the learned policy exhibits a strong global tendency to guide the agent towards the goals efficiently. Moreover, agents intentionally choose 'safer' paths to avoid penalties when they are close to an obstacle.

This observation highlights a fundamental feature of Q-learning:

*Even when actions have uncertain outcomes, Q-learning can still converge to an approximately optimal policy by maximizing expected cumulative rewards across episodes.*

**Q2(d)**

The performance and convergence behaviour of the Q-learning algorithm are strongly influenced by three key iteration parameters:

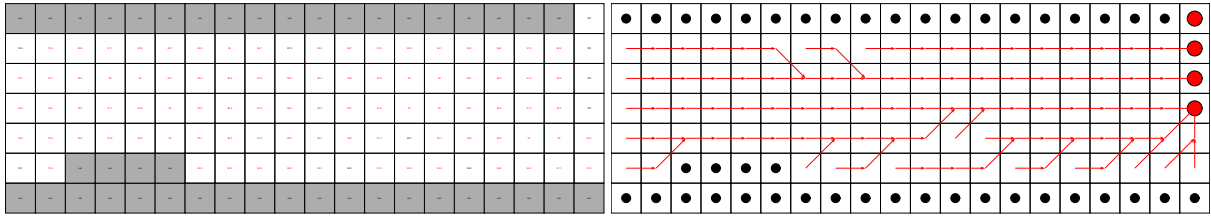
- **Total number of training loops i:** This refers to how many times the policy learning process (`find_policy()`) is called. Increasing this value allows more opportunities for the policy and value function to improve, especially in early stages of learning. Also the variable  $\epsilon$  in our implementation follows the decay schedule:

$$\epsilon(i) = \frac{1}{\sqrt{1 + 0.25i}} \quad (11)$$

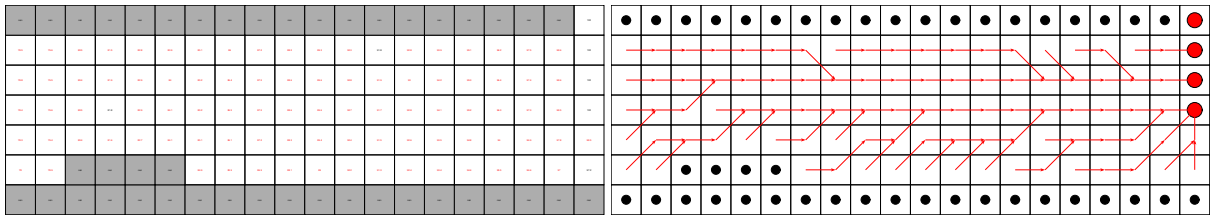
where  $i$  is the total number of training iterations. Initially,  $\epsilon = 1$ , encouraging full exploration. As  $i$  increases,  $\epsilon$  gradually decreases, leading the policy toward exploitation of the learned value function.

- **Number of episodes per loop:** Each loop typically involves a batch of episodes. A higher number of episodes per loop allows the algorithm to gather more experience before updating the policy, which may improve stability and convergence but increase computation.
- **Maximum steps per episode:** This defines the horizon of each episode. A small value may cause early termination before useful rewards are discovered, while a large value allows deeper exploration but increases variance. (But in order not to break the integrity of the overall code, this part I will not consider in this question. Because the setting of this parameter is in the built-in code (`episode_sampler.py` `max_steps = 2000`) and cannot be modified by calling the method directly.)

But during the experiment I found a serious problem, the effect of changing the parameters on the final result is not very straightforward to see, as shown in **Figure 18** and **Figure 19**. It looks even better than 200 when the number of iterations is equal to 32. This is clearly counter-intuitive, as the final result must gradually move closer to the optimal solution as the number of q-learning iterations increases.



**Figure 18:** Final result of value function and policy with iteration number  $i = 32$

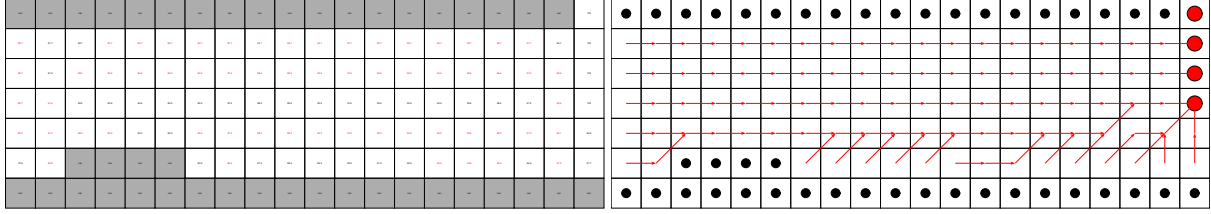


**Figure 19:** Final result of value function and policy with iteration number = 200

Upon observation, I noticed another stochasticity that was also mentioned in the previous question: **the stochasticity of the robot's movement**. This will make the final result of each experiment always full of randomness and difficult to make comparisons. To clear up this confusion, I decided to change the robot's movement to be predictable for the time being. While this may go against the setting of this scenario, it was necessary to find the effect of the number of iterations on Q-learning. (While setting a **fixed seed** can also eliminate this randomness, this is relatively more complex.)

```
1 env.set_nominal_direction_probability(1)
2
```

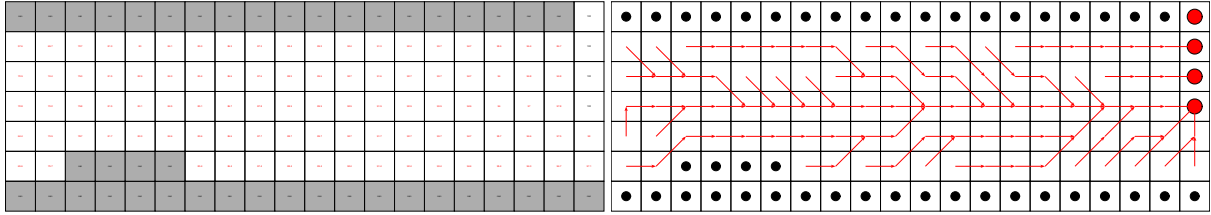
Here, we use the original data (**iteration number  $i = 40$ , episode  $e = 32$** ) as a reference (*Figure 20*). Control variables are applied and analysed by varying  $i$  and  $e$  respectively.



**Figure 20:** Final result of value function and policy,  $e=32$   $i=40$  (original)

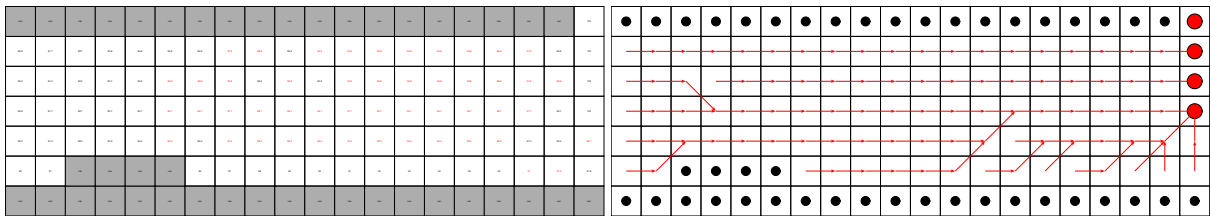
- **The effect of the total number of iterations  $i$  (episode  $e = 32$ ):**

- **When iteration number = 5: (Figure 21)** From the final result, it is visually clear that the overall strategy is not good. Most of the chosen paths are suboptimal. However, the general direction still leads to the goal without getting stuck or backtracking, which indicates that the Q-learner has started to search for the optimal route. The limited number of iterations, however, prevents the policy from fully converging to the optimal solution. This can also be observed from the value function visualization: the overall values are significantly lower compared to the reference case, suggesting that the agent is still in the early learning stage, with many suboptimal decisions yet to be refined.



**Figure 21:** Final result of value function and policy,  $e=32$   $i=5$

- **When iteration number = 100: (Figure 22)** We observe that the policy is now nearly perfect (except for a small unexplained behavior on the left side, which might be due to randomness). The value function also closely matches our reference case. Although there are still minor differences in the local strategies, simple calculations reveal that both the current and reference policies are in fact optimal—just following different but equally short paths. This indicates that the Q-learner has already converged to an optimal solution by around 32 episodes, and further iterations beyond this point do not bring significant improvements.



**Figure 22:** Final result of value function and policy,  $e=32$   $i=100$

- **The effect of the total number of episode  $e$  (iteration  $i = 40$ ):**

- **When episode number = 5: (Figure 23)** With a larger number of episodes, the Q-learner has gained substantial experience interacting with the environment. The resulting policy is smooth and highly directed—most states lead the agent efficiently toward the nearest terminal goal. While we observe that the final policy contains multiple different routes from the same starting region, this is not an error. Instead, these variations represent multiple equally optimal solutions—all of them have the same total path cost or length. This highlights the fact that the Q-learning algorithm is capable of discovering a diverse set of optimal strategies when they exist, depending on the stochastic nature of the learning process.

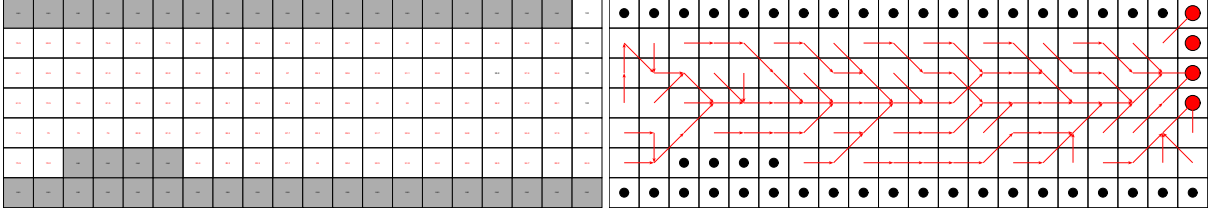


Figure 23: Final result of value function and policy,  $e=32$   $i=40$

- **When episode number = 100: (Figure 24)** In contrast, with more episodes, the Q-learner gathers much more experience. The resulting policy is significantly smoother and more directed—almost all paths clearly point to the nearest terminal goal. While there may still be some slight irregularities (possibly due to randomness in action selection or state transitions), the majority of the grid shows confident and optimal directions. The value function is also smoother and forms consistent gradients toward the terminal states, showing that the agent has learned to associate appropriate expected returns with each state.

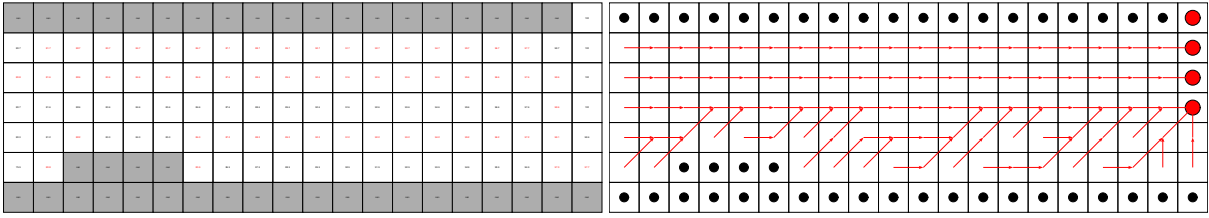


Figure 24: Final result of value function and policy,  $e=100$   $i=40$

### Conclusion:

Through experimental observations, I found that the **most critical parameter influencing the Q-learner's performance is the number of episodes**. In contrast, the total number of iterations primarily serves to accelerate training—though it can also affect final performance.

In an idealized setting, Q-learning assumes an **infinite number of episodes**, ensuring that every state–action pair is visited sufficiently and the value estimates fully converge. However, in practice, such exhaustive exploration is infeasible due to the **enormous computational and time costs** required for very large episode counts.

As discussed in Sutton and Barto's book, a practical solution is to **introduce a finite number of iterations**, each consisting of multiple episodes. After each iteration, the agent updates its value estimates and policy, while also **gradually reducing the exploration rate** ( $\epsilon$ ). This compromise ensures convergence to a near-optimal policy while significantly **reducing computational resources**.

To validate this conclusion, I tested two extreme configurations:

- A very large number of episodes (e.g., 500) with only 5 iterations (*Figure 62*).
- A very small number of episodes (e.g., 5) with 500 iterations (*Figure 63*).

(Visualizations for these cases are included in the appendix.)

The results show that:

- With a **high episode count**, even a single iteration is often sufficient for the agent to find the optimal path.
- With **low episode counts**, the Q-learner fails to converge to the optimal policy. As  $\epsilon$  decays with more iterations, the agent becomes less exploratory and eventually converges to a **random suboptimal policy**.

This supports the claim that **sufficient episode exploration is key** to effective learning in Q-learning, while iterations and  $\epsilon$  decay help balance exploration and exploitation over time.

**Q2(e)**

The complete algorithm for this Q-learner is as follows:

**Algorithm 2** Q-learning with Multi-level Loop Control

---

```

1: Initialize  $Q(s, a)$  arbitrarily
2: Initialize policy  $\pi$  as  $\varepsilon$ -greedy with respect to  $Q$ 

3: for iteration  $i = 0$  to  $N_{\text{iteration}}$  do                                ▷ Outer loop
4:   Update  $\varepsilon \leftarrow \frac{1}{\sqrt{1+0.25 \cdot i}}$ 

5:   for episode  $e = 0$  to  $N_{\text{episodes}}$  do                                ▷ Middle loop
6:     Initialize  $S$ 
7:     Choose  $A$  from  $S$  using behaviour policy  $b$  (e.g.,  $\varepsilon$ -greedy)

8:     for step  $t = 0$  to  $N_{\text{max\_steps}}$  do                                ▷ Inner loop
9:       Take action  $A$ , observe reward  $R$  and next state  $S'$ 
10:       $Q(S, A) \leftarrow Q(S, A) + \alpha \left[ R + \gamma \max_a Q(S', a) - Q(S, A) \right]$ 
11:       $S \leftarrow S'$ 
12:      Update Q-table and policy

13:      if  $S$  is terminal then
14:         $Q(S, A) \leftarrow R$ 
15:        Update Q-table and policy
16:        break
17:      end if

18:    end for                                ▷ Inner loop end

19:  end for                                ▷ Middle loop end

20: end for                                ▷ Outer loop end

```

---

**Analysis Strategy:**

I argue that comparing episodes within the same iteration (outer loop) is not meaningful. This is because the algorithm employs the *exploring start* method, meaning that the starting position of each episode is entirely random. In some cases, the agent might even start at or very close to the terminal state (as can be observed in the `LowLevelEnvironment()` method).

This introduces a high degree of randomness into the final outcome of each episode. Therefore, in this analysis, I will focus on comparing episodes across different iterations instead. This approach allows averaging across multiple episodes in one iterations, thereby reducing the variance caused by randomness in the starting positions.

**Analysis method:**

From the algorithm, we can find the only thing that affects the running time of one episode is **the number of steps the agent takes to reach the terminal from a random position**. In this analysis, I will not discuss external factors such as CPU performance, as they are uncontrollable variables.

To visualize both the execution time and the number of steps per episode, I made some minor modifications to the code. Specifically, in the `EpisodeSampler()` method, I added functionality to record the number of steps used in each episode and implemented a new method to retrieve this value directly. However, since the program includes an experience replay buffer, I also added some new code to the `TDController()` class to ensure proper handling. In the main program (`q3_e.py`), I calculated the average number of steps across all episodes for each iteration and introduced a timer to measure the duration of each iteration. I have labelled all the changes and you can view the changes directly from the code in the folder.

Finally, I generated visual plots based on these values to analyze the results.



(Figure 25) is the result with initial parameters: adaptive  $\varepsilon$  and 32 episodes per iteration. I will define this as the standard case below. We can clearly observe that **as the number of iterations increases, both the number of steps per episode and the execution time tend to decrease**. In the early stages, there is a sharp decline. This is because the agent gradually discovers a path to the goal, rather than endlessly exploring until reaching the maximum step limit. In the later stages, the data stabilizes at relatively low values, indicating that the agent has developed a more efficient policy and is consistently following near-optimal paths. There are two main factors influencing the number of steps (time) per episode: **the exploration rate  $\varepsilon$  and the strategy policy  $\pi$** .

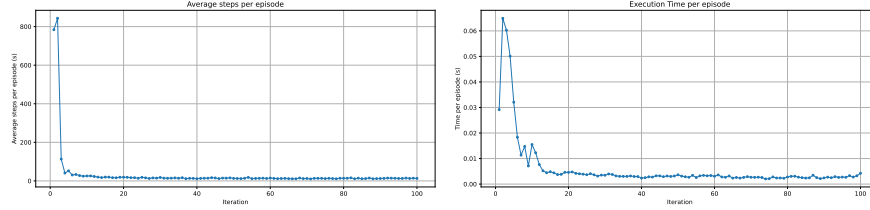


Figure 25: (a)Average steps per episode (b)Average Execution time per episode

- **Exploration rate  $\varepsilon$ :** In the provided code, the exploration rate  $\varepsilon$  is reduced after each outer iteration. This leads the agent to gradually favor a fixed path rather than wandering randomly without direction. Although the final path may converge to a suboptimal route if the number of episodes is insufficient, the number of **steps (time) required per episode will still decrease steadily** as the standard. Since I set this experiment to have only 3 episodes per iteration, the data has large fluctuations. This is because the results are still affected by randomness. (Figure 26)

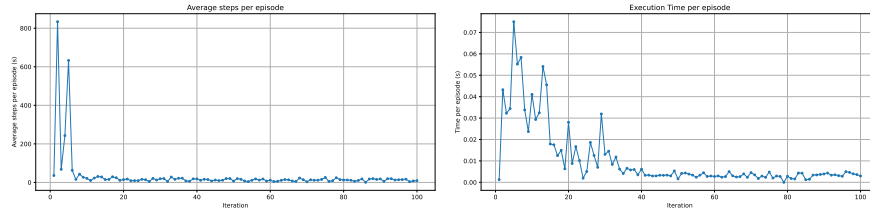


Figure 26: 3 episode per iteration (a)Average steps per episode (b)Average Execution time per episode

- **Strategy policy  $\pi$ :** Even when the exploration rate  $\varepsilon$  remains fixed and less than 1, **the time and steps in an episode will still decrease as trend**. This is because, under an  $\varepsilon$ -greedy policy, the agent selects the action with the highest estimated Q-value most of the time. As a result, the agent tends to move toward the goal, even if occasional exploration causes temporary deviations from the optimal path. This behavior ensures that the overall direction remains efficient, though individual trajectories may vary slightly. In this experiment, the results show a significantly smoother trend compared to the previous case. This is attributed to using a fixed exploration rate of  $\varepsilon = 0.8$  and averaging the performance of 32 episodes per iteration, which helps reduce variance due to randomness. However, it's also worth noting that in the early iterations, the fixed exploration rate leads to fewer exploratory actions than the standard, resulting in shorter episodes and lower computation time. As training progresses, the fixed  $\varepsilon$  causes the agent to continue exploring even after learning a near-optimal policy. This leads to longer episodes with more fluctuations in performance and timing, in contrast to a decaying  $\varepsilon$  that would stabilize the agent's behavior. (Figure 27)

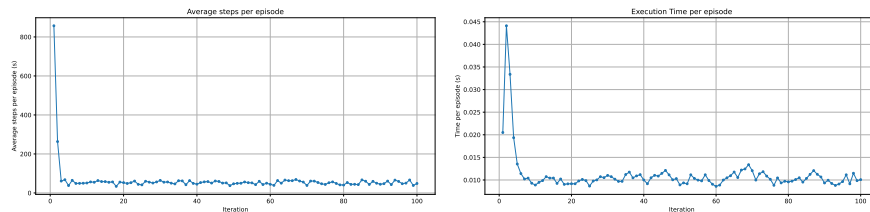


Figure 27: Fixed  $\varepsilon = 0.8$  (a)Average steps per episode (b)Average Execution time per episode

**Q2(f)**

**We introduced the on-policy method in Q1(a):** An on-policy method evaluates and improves the same policy that is used to generate the behavior. The learning updates are made with respect to the behavior policy  $\pi$ , which is often  $\epsilon$ -greedy. The full return is computed as *Equ 1*.

**We introduced the TD(0) method in Q1(d):** Temporal-Difference learning with zero-step lookahead is a method that updates the value of a state using the observed reward and the estimated value of the next state. It performs updates after every step without waiting for the end of the episode. The update rule was shown in *Equ 6*.

**SARSA** is an *on-policy* Temporal-Difference (TD) control algorithm that estimates the action-value function  $q_\pi(s, a)$  with respect to the current policy  $\pi$ . It learns from actual actions taken and improves the same policy used for behavior. The update rule for TD(0) in SARSA is given by:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \quad (12)$$

where:

- $Q(S_t, A_t)$ : current estimate of the action-value.
- $R_{t+1}$ : reward after taking action  $A_t$  in state  $S_t$ .
- $\gamma$ : discount factor.
- $\alpha$ : learning rate.
- $Q(S_{t+1}, A_{t+1})$ : estimate of next state's action-value from the current behavior policy.

**This rule reflects the on-policy nature of SARSA because the update uses the next action  $A_{t+1}$  selected by the current  $\epsilon$ -greedy policy  $\pi$ , not the greedy action with maximum  $Q$ -value.**

**Table 6:** Comparison between SARSA (On-policy) and Q-learning (Off-policy)

| Criterion                       | SARSA (On-policy)                                               | Q-learning (Off-policy)                                               |
|---------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------|
| Policy type                     | Learns using current behavior policy $\pi$                      | Learns with respect to greedy target policy                           |
| Risk handling                   | Tends to be safer in risky environments due to policy alignment | Might learn risky paths since exploration and update policies differ  |
| Convergence speed               | Slower, as updates are based on actual chosen actions           | Usually faster due to aggressive updates toward maximum Q             |
| Optimality (eventually)         | May converge to sub-optimal policies under limited exploration  | More likely to converge to optimal policy with sufficient exploration |
| Stability in noisy environments | More stable, as actions reflect actual behavior                 | Less robust, sensitive to variance in behavior policy                 |

**Conclusion:**

As shown in Table 6, without a specific environment context, there is essentially no absolute advantage of one algorithm over the other. SARSA maintains consistency between the behavior policy and the update process, **making it more stable and safer**. It tends to perform better in noisy or high-risk environments. However, in scenarios that require extensive exploration, this advantage can turn into a disadvantage, as SARSA **typically converges much more slowly and may even converge to a suboptimal policy**.

## Q2(g)

Our implementation follows the core structure and update logic outlined in the pseudocode presented by **Sutton and Barto**:

**Sarsa (on-policy TD control) for estimating  $\pi \approx \pi_*$** 

**Algorithm parameters:** step size  $\alpha \in (0, 1]$ , small  $\varepsilon > 0$

**Initialize**  $Q(s, a)$  arbitrarily for all  $s \in S^+$ ,  $a \in \mathcal{A}(s)$  except that  $Q(\text{terminal}, \cdot) = 0$

**Loop** for each episode:

- Initialize  $S$
- Choose  $A$  from  $S$  using a policy derived from  $Q$  (e.g.,  $\varepsilon$ -greedy)
- **Loop** for each step of episode:
  - Take action  $A$ , observe reward  $R$ , next state  $S'$
  - Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.,  $\varepsilon$ -greedy)
  - $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$
  - $S \leftarrow S'$
  - $A \leftarrow A'$
- until  $S$  is terminal

- **Set the initial state  $S$ , action  $A$ , reward  $R$  at the start of the episode:**

```

1  # state
2  s = episode.state(0)
3  # state coordinates
4  coords = s.coords()
5  # reward
6  reward = episode.reward(0)
7  # action
8  a = episode.action(0)
9

```

- **Loop for each step of episode:**

```

1  for step_count in range(1, episode.number_of_steps()):
2

```

- **Get next state  $S'$  and its coordinates**

```

1  s_next = episode.state(step_count)
2  coords_next = s_next.coords()
3

```

- **Find next action  $A'$  :**

```

1  a_prime = episode.action(step_count)
2

```

- **Calculate next Q value  $Q(S', A')$  :**

```

1  a_prime = episode.action(step_count)
2

```

- **Calculate the new Q value  $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$ :**

```

1  # Define the current Q-value Q(S,A)
2  current_q = self._Q[coords][a]
3  # Calculate the new Q-value
4  new_q = current_q + self._alpha * (reward + self._gamma * next_q -
5  current_q)

```

- **Update the the grid:**

```

1  self._update_q_and_policy(coords, a, new_q)
2

```

- Get the current state, action, reward for the next episode  $S \leftarrow S', A \leftarrow A', R \leftarrow R'$ :

```

1     reward = episode.reward(step_count)
2     s = episode.state(step_count)
3     coords = s.coords()
4     a = episode.action(step_count)
5

```

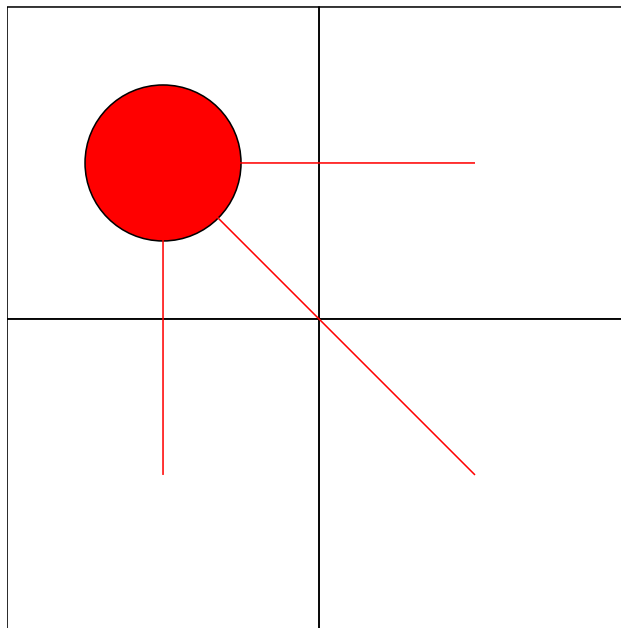
- Set the final Q-value =  $R$  (action is terminate):

```

1     new_q = reward
2     self._update_q_and_policy(coords, a, new_q)
3

```

This completes one full pass of the Sarsa update over a single episode.



**Figure 28:** Illustration of agent action selection in a grid environment.

**Figure 28** is a visual representation of the strategies learned after SARSA convergence. The red circle is the target endpoint; the red lines are the greedy actions (strategies) chosen by the agent in different states, guiding it on how to move to the endpoint. This has proved that SARSA agent succeeded in getting the optimal policy.

## Q2(h)

## Static analyze:

## • State Values

- **Q-learning (Figure 29, left):** The state values appear higher and more uniformly propagated across the corridor, especially along the optimal route. This suggests that Q-learning is optimistic in its estimates and aggressively learns the best possible value.
- **SARSA (Figure 30, left):** The state values are generally lower and more conservative. This reflects SARSA's on-policy nature, which updates values based on the actual actions taken (including exploratory ones), often leading to more cautious estimates.

## • Extracted Policies

- **Q-learning (Figure 29, right):** The arrows clearly show a **direct and optimal path** to the goal (red circles). The agent quickly converges to a greedy path that avoids most unnecessary movements and risky states. As stated in the previous questions, although some state actions will deviate slightly from the optimal path, this is due to randomness in the system and is acceptable.
- **SARSA (Figure 30, right):** The extracted policy is more **conservative and circuitous - in other words, far from ideal**. Initially, I suspected that the SARSA algorithm might not have been functioning correctly, as the results deviated significantly from expectations. However, after thoroughly reviewing the code, verifying the algorithm, and experimenting with various hyperparameters, I still could not obtain a more reasonable outcome. I eventually concluded that this behavior stems from the on-policy nature of SARSA, the agent seemingly taking safer or longer routes and avoiding the edges more often. While SARSA may perform better in high-risk environments, **it appears less suitable in this scenario**, where the objective is to guide a robot to find the shortest path in an airport environment.

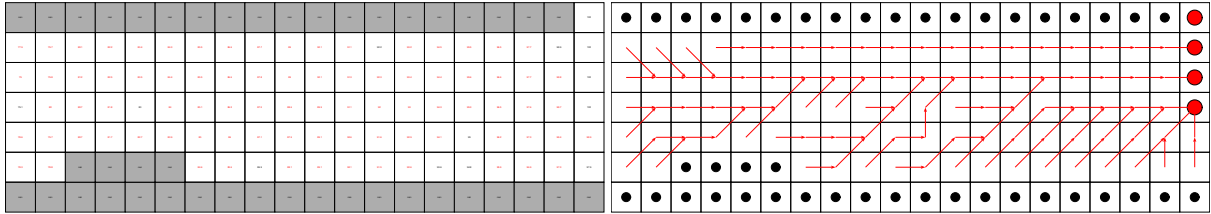


Figure 29: Final result of value function and policy from Q-learning

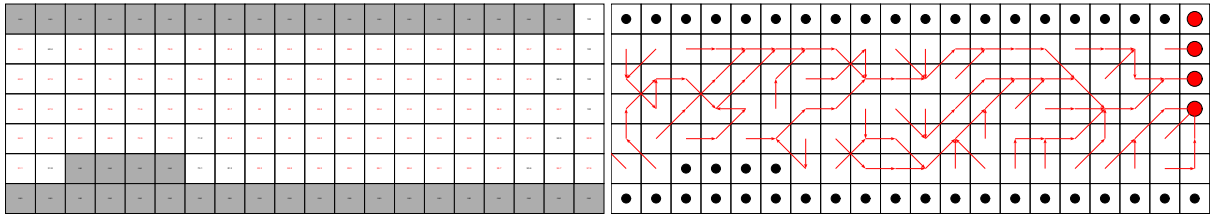
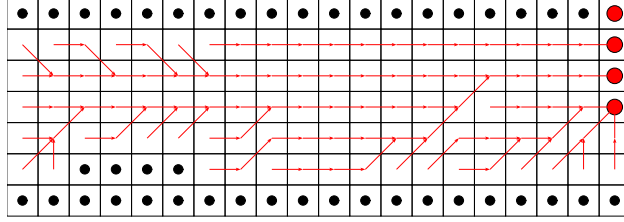
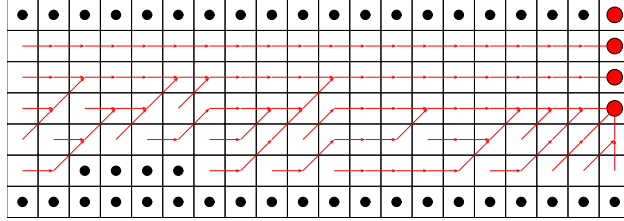


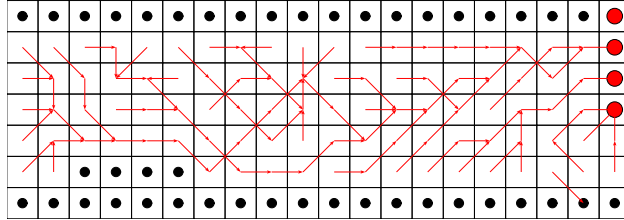
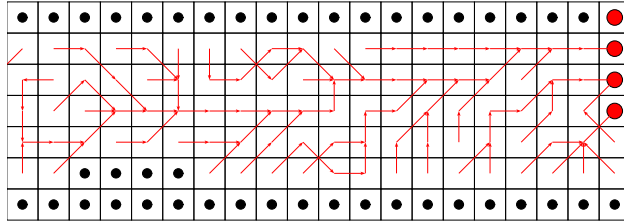
Figure 30: Final result of value function and policy from SARSA

**Dynamic analyze:**

- **Q-learning** converges to the optimal solution by approximately the 30<sup>th</sup> iteration (**Figure 31**), with only minor differences compared to the result at the 500<sup>th</sup> iteration (**Figure 32**). In later iterations, only a small number of state-action pairs continue to be updated.

**Figure 31:** Final result of value function and policy from Q-learning at the 30th iteration**Figure 32:** Final result of value function and policy from Q-learning at the 500th iteration

- **SARSA**, in contrast, shows significant differences between the 30<sup>th</sup> (**Figure 33**) and 500<sup>th</sup> iterations (**Figure 34**), with most state-action pairs being updated in every iteration. This is because SARSA, as an on-policy algorithm, updates its Q-values based on the actual actions taken under the current policy, which includes exploratory actions. Therefore, the learning process continues to adapt dynamically as the policy itself evolves.

**Figure 33:** Final result of value function and policy from SARSA at the 30th iteration**Figure 34:** Final result of value function and policy from SARSA at the 500th iteration**Conclusion:**

**Q-learning** learns an optimal and efficient policy quickly by relying on greedy updates, resulting in higher and more optimistic state values. In contrast, **SARSA** learns more conservatively due to its on-policy nature, taking exploratory actions into account. This leads to safer but longer paths. Dynamically, Q-learning converges early, while SARSA continues to update over many iterations as its policy evolves. **Overall, Q-learning performs much better than SARSA in this scenario.**

## References

## Appendix

## A Q1(b) Additional Visualisations

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 83.7 | 83.8 | 83.9 | 83.9 | 83.9 | 87.5 | 88   | 88   | 87.9 | 88.1 | 88.9 | 88.1 | 88.9 | 88.9 | 87.7 |
| 88.2 | 88.9 | 88.9 | 88.9 | 87.7 | 88.1 | 88.7 | 88.7 | 87.9 | 88   | 88.1 | 88.9 | 88.2 | 87.9 | 88.8 |
| 82.7 | 83.9 | 88.2 | 88.2 | 87.2 | 88.4 | 88   | 87.2 | 82.7 | 83.9 | 88   | 88.2 | 87.9 | 88.9 | 100  |

Figure 35: On-Policy MC First Visit ( $N = 50$ )

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 83.9 | 88.1 | 88.9 | 88   | 87   | 87.9 | 88.9 | 88.1 | 87.9 | 88.1 | 88.9 | 88.9 | 88.9 | 87.9 |      |
| 88.9 | 88.1 | 88.9 | 88.9 | 87.9 | 88.9 | 88.9 | 88.9 | 87.9 | 88   | 88.1 | 88.9 | 88.9 | 87.9 | 88.7 |
| 83.9 | 88   | 88.9 | 88.9 | 88.9 | 88.9 | 88.9 | 88.9 | 87.9 | 82.7 | 83.9 | 88.1 | 88.9 | 87.9 | 100  |

Figure 36: On-Policy MC Every Visit ( $N = 50$ )

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 79.6 | 83.9 | 88.9 | 87.9 | 88.7 | 88.9 | 88.9 | 87.7 | 88.7 | 88.9 | 88.7 | 88.7 | 88.9 | 87.9 | 87.7 |
| 8    | 88   | 8    | 87.2 | 88.2 | 88.4 | 88.9 | 88.9 | 88.1 | 88.1 | 88.7 | 88.9 | 87   | 87.9 | 88.9 |
| 88.9 | 88.1 | 87   | 87.9 | 88.9 | 88.9 | 87   | 87.9 | 88.1 | 88.9 | 88.9 | 87.9 | 88.9 | 88.9 | 100  |

Figure 37: Off-Policy MC First Visit ( $N = 200$ )

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 8    | 88.9 | 88.7 | 88   | 88.1 | 88.1 | 88.7 | 87.9 | 88.9 | 88.9 | 88.7 | 88.9 | 88.1 | 87.1 | 87.9 |
| 88.9 | 8    | 8    | 87.2 | 88.9 | 88   | 88.9 | 87.9 | 100  | 88.9 | 88.9 | 87.2 | 88.1 | 88.9 |      |
| 88   | 88.2 | 88.9 | 88.9 | 88   | 88.9 | 88.9 | 87.9 | 88.1 | 88.1 | 88.9 | 88.9 | 87.7 | 88.9 | 100  |

Figure 38: Off-Policy MC Every Visit ( $N = 200$ )

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 83.4 | 88.4 | 88.9 | 88.4 | 87.9 | 88.4 | 88.9 | 88.9 | 87.9 | 88.4 | 88.7 | 88.4 | 88.9 | 87   | 87.7 |
| 83.9 | 88.9 | 8    | 88.2 | 88.9 | 88.4 | 88.9 | 87.9 | 88.9 | 88.9 | 88   | 88.9 | 88.9 | 87.7 | 88.9 |
| 88   | 88.1 | 87.7 | 87.9 | 88.9 | 88.9 | 88.9 | 87.9 | 88.1 | 88.1 | 88.9 | 87.7 | 88.9 | 88.9 | 100  |

Figure 39: Off-Policy MC First Visit ( $N = 1000$ )

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 88.9 | 88.9 | 88.1 | 87   | 88.9 | 88.9 | 88.7 | 87.9 | 88.9 | 88.9 | 88.9 | 88.9 | 88.9 | 87.1 | 87.9 |
| 88.1 | 88.1 | 8    | 87.9 | 88.4 | 88.9 | 88.9 | 87.7 | 88.9 | 88.9 | 88.1 | 88.1 | 87   | 88.9 | 88.9 |
| 88.9 | 88.7 | 88.9 | 87.9 | 88.7 | 88.7 | 88.9 | 87.7 | 88.9 | 88.9 | 88.9 | 87.7 | 88.9 | 88.9 | 100  |

Figure 40: Off-Policy MC Every Visit ( $N = 1000$ )

## B Q1(f) Value Visualisation

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 81.9 | 83.5 | 84.7 | 85.8 | 85.9 | 88   | 88.1 | 90.1 | 91.2 | 92.3 | 93.3 | 94.4 | 95.4 | 96.4 | 97.4 |
| 79.4 | 79.6 | 88.9 | 86.4 | 87.5 | 88.7 | 88.8 | 90.9 | 92.1 | 93.2 | 94.3 | 95.4 | 96.5 | 97.6 | 98.7 |
| 82.5 | 84.1 | 85.4 | 86.5 | 87.7 | 88.9 | 90.1 | 91.3 | 92.5 | 93.7 | 94.9 | 96.2 | 97.4 | 98.7 | 100  |

Figure 41: Ground Truth

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 83.4 | 79.1 | 88.9 | 88.9 | 88.7 | 87.7 | 88.7 | 88.1 | 91.3 | 92.1 | 93.4 | 94.1 | 95.9 | 96.7 | 97.9 |
| 88.9 | 87.2 | 88.9 | 88.1 | 87.9 | 88.9 | 88   | 87.2 | 92.2 | 93.2 | 94.9 | 96.7 | 96.9 | 97.9 | 98.9 |
| 88.9 | 77.2 | 88.1 | 88.9 | 87.9 | 88.9 | 88.1 | 91.3 | 93.9 | 94.9 | 96   | 96.1 | 97.9 | 98.7 | 100  |

Figure 42:  $\alpha = 0.01$  ( $N=120$ )

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 87.2 | 88.9 | 88.7 | 88.9 | 88.9 | 87.9 | 88   | 88   | 91.2 | 92.1 | 93.1 | 94.9 | 95.9 | 96.9 | 97.9 |
| 79.9 | 79.1 | 88.9 | 88.9 | 87.9 | 88.9 | 88.9 | 88.9 | 92.1 | 92.2 | 94.1 | 96.9 | 96.9 | 97.7 | 98.7 |
| 88.1 | 88.9 | 88.9 | 88.9 | 87.9 | 88   | 88.1 | 91.3 | 92.7 | 93.9 | 94.9 | 96.2 | 97.9 | 98.9 | 100  |

Figure 43:  $\alpha = 0.01$  ( $N=240$ )

|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 83.4 | 88.7 | 88.9 | 88.9 | 88.9 | 88   | 88   | 88   | 91.1 | 92.7 | 93.3 | 94.1 | 95.2 | 96.1 | 97.2 |
| 79.9 | 88.1 | 88.9 | 88.9 | 87.9 | 88.7 | 88.9 | 88.9 | 88   | 91.1 | 94.9 | 96.1 | 96.9 | 97.2 | 98.9 |
| 87.9 | 88.1 | 88.9 | 88.9 | 87.9 | 88.7 | 88.1 | 91.3 | 93.9 | 94.7 | 96   | 96.7 | 97.2 | 98.9 | 100  |

Figure 44:  $\alpha = 0.01$  ( $N=360$ )

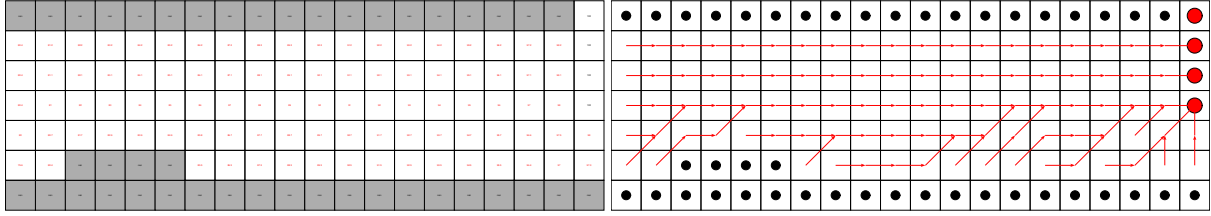
|      |      |      |      |      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 88.7 | 88.9 | 88.9 | 88   | 87.1 | 88   | 88   | 88.1 | 91.2 | 92.1 | 93.9 | 94.9 | 95.9 | 96.9 | 97.7 |
| 79.9 | 79.1 | 88.9 | 88.9 | 87.9 | 88.9 | 88.9 | 88.9 | 91.1 | 91.1 | 92.2 | 94.9 | 96.9 | 97.9 | 98.9 |
| 88.1 | 88.2 | 88   | 88.1 | 87.9 | 88.9 | 88.9 | 88   | 91   | 91   | 93.9 | 94.9 | 95.9 | 97.2 | 98.9 |

Figure 45:  $\alpha = 0.01$  ( $N=400$ )

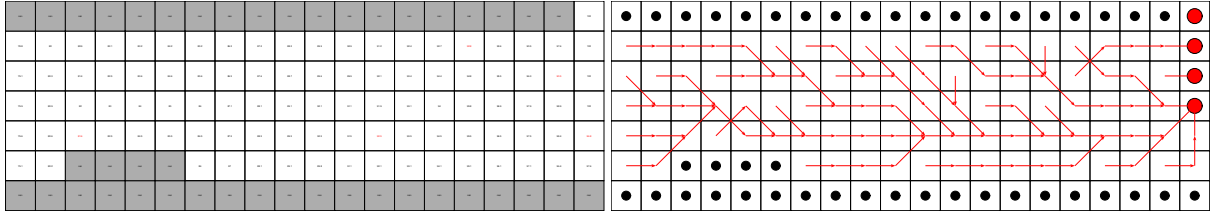




### C Q2(d) Two more situation



**Figure 62:** Final result of value function and policy,  $e=500$   $i=5$



**Figure 63:** Final result of value function and policy,  $e=5$   $i=500$